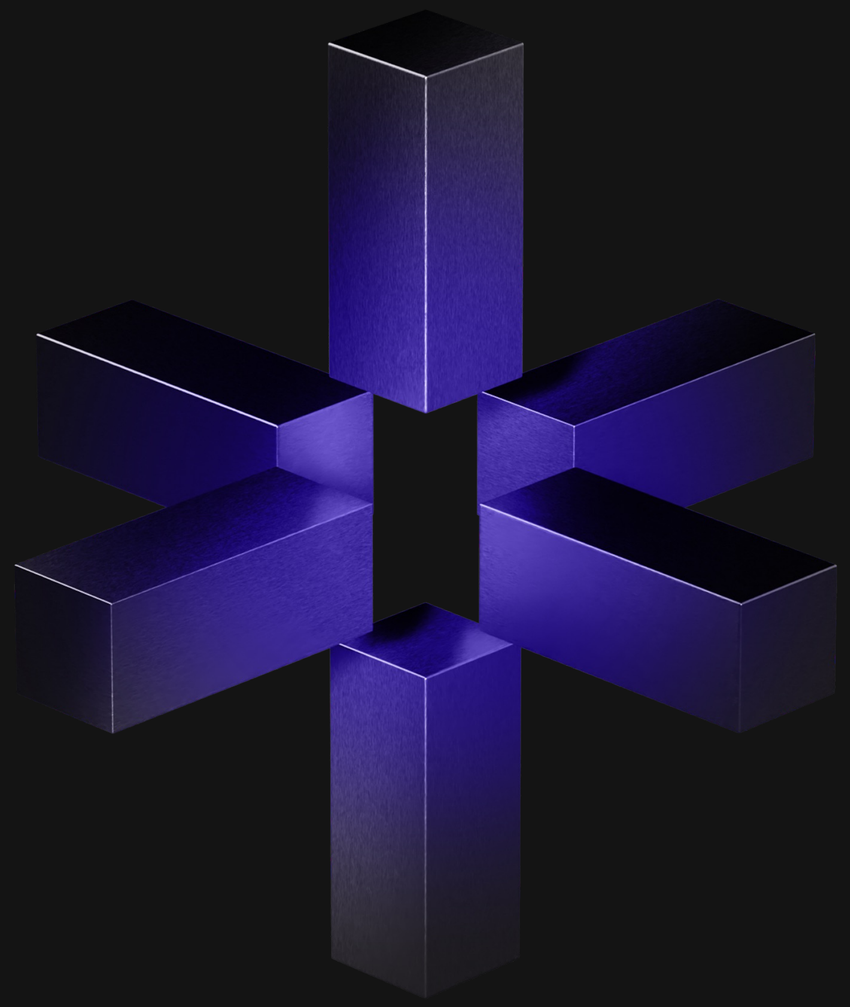
ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

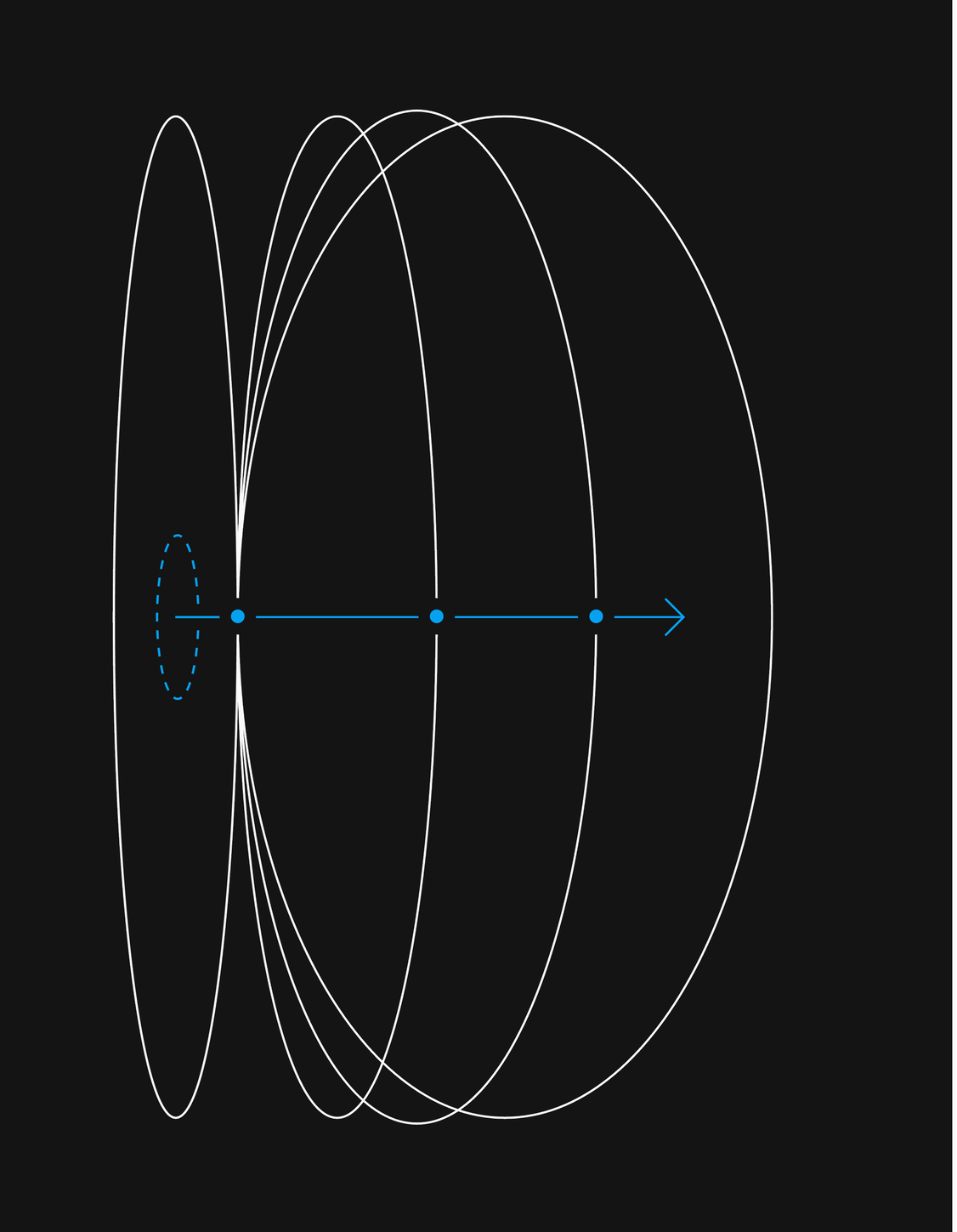


Контроль качества и выбор модели

ML

Машинное обучение

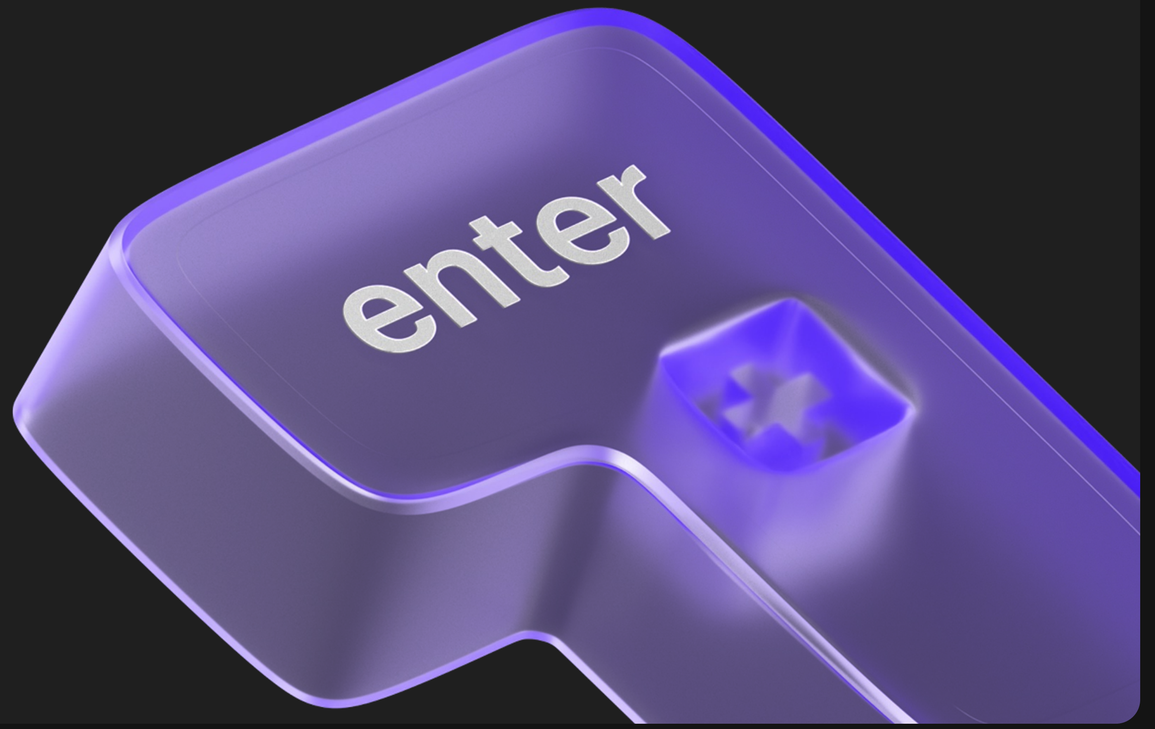
02



План лекции

- Способы контроля качества (кросс-валидация)
- Способы управления разбиением данных (перемешивание, группировка, стратификация)
- Кросс-валидация на временных рядах
- Подбор гиперпараметров

Способы контроля качества

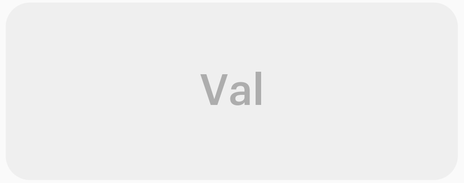


Основа любого контроля качества

Разбиваем выборку на две части: обучающую и тестовую (контрольную).



Позже рассмотрим разбиение с учётом валидации, но пока остановимся на двух подвыборках.

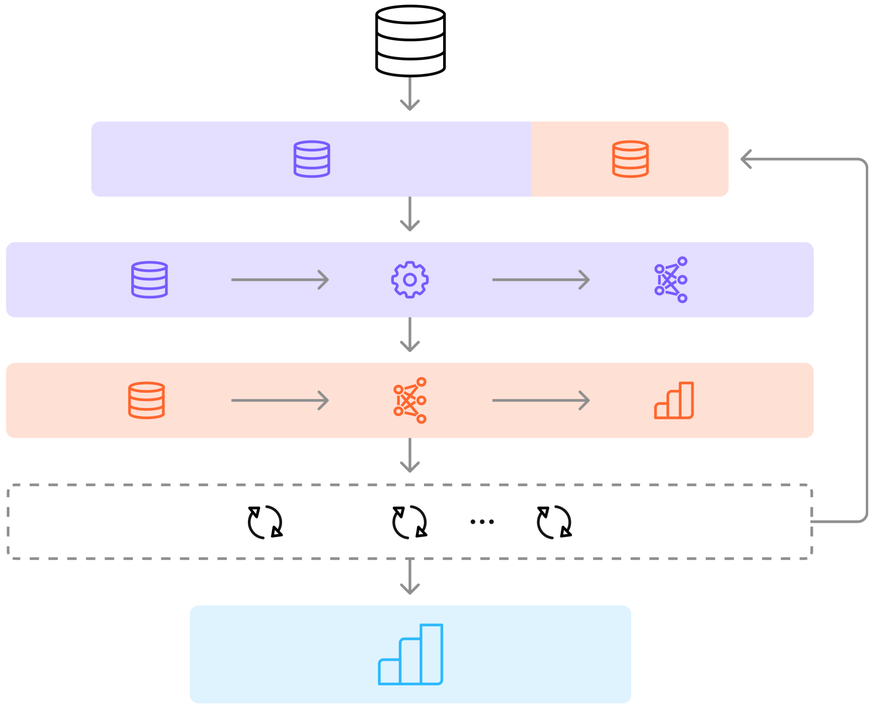


Без правильного разделения данных метрика может быть обманчивой. Например, если мы случайно «подглядываем» в тестовую выборку при обучении, метрики покажут отличные результаты, но на новых данных модель провалится. Такой эффект называется утечкой данных (Data Leakage).

Кросс-валидация

Это метод оценки качества модели машинного обучения, при котором данные многократно разделяются на обучающую и тестовую выборки, чтобы проверить, насколько хорошо модель обобщает результаты на новых данных.

1. Разделить датасет на две части: обучающую и тестовую.
2. Обучить модель на обучающей выборке.
3. Оценить её на тестовой выборке.
4. Повторить эти шаги несколько раз в зависимости от выбранного типа кросс-валидации.



Способы контроля

Многие источники относят Hold-Out к методам простой валидации, не считая его частным случаем кросс-валидации (даже в предельно упрощённом виде).

Отложенный контроль

Held-Out Data,
Hold-Out Set

Скользящий контроль

Cross-Validation — иногда так называют «всё»

Бутстрэп

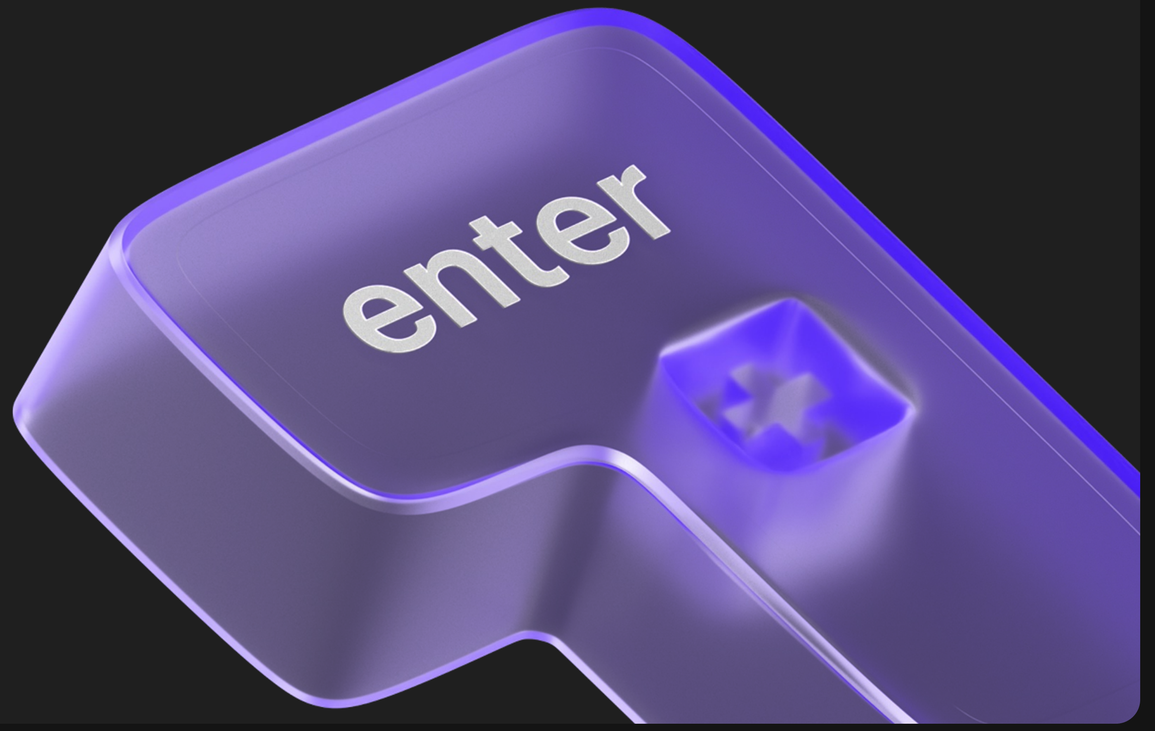
Bootstrap

Контроль по времени

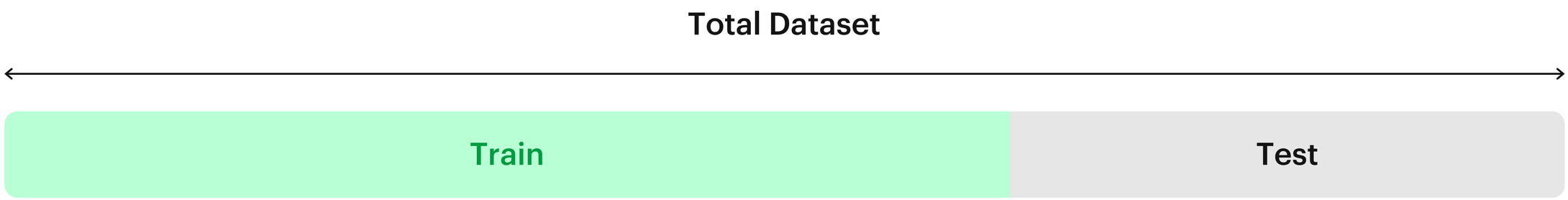
Out-of-Time-контроль

Далее мы подробно разберём разные способы контроля

Отложенный контроль
Hold-Out Set



Hold-out



данные, используемые для обучения модели

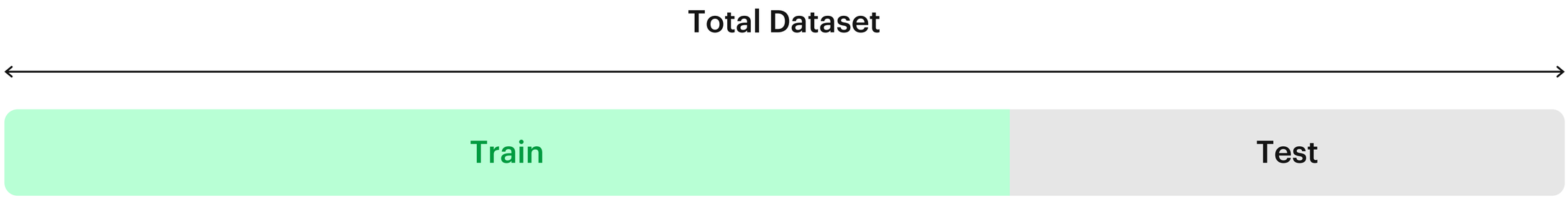
данные, используемые только для финальной оценки модели (будто это новые реальные данные)

Обычно делим 80% (Train) / 20% (Test).

Больше тест — надёжнее оценка.

До финальной оценки качества модели тестовая выборка никак не используется (например, не для EDA, бейзлайна или экспериментов).

Hold-out



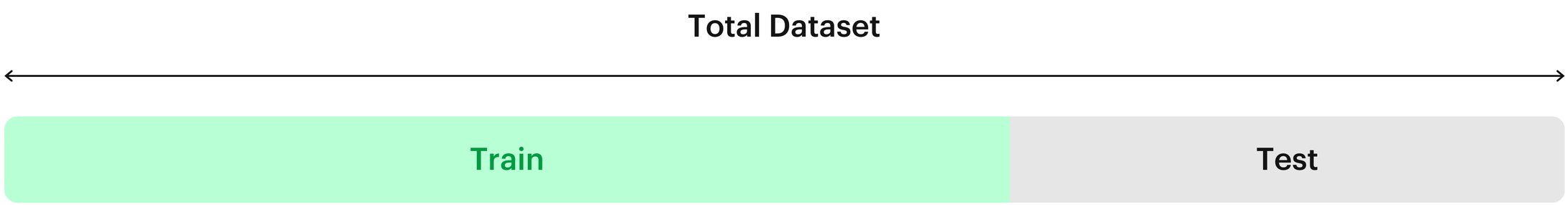
Оценка ошибки зависит от выбора конкретной отложенной выборки.

Часто сильно меняется при её смене.

Если переобучить алгоритм для всех данных, то мы не узнаем оценку его ошибки.

В каком-то смысле это неустранимый недостаток.

Hold-out




```
train_test_split(X, y, test_size=0.05, random_state=0)   # → accuracy: 0.872
train_test_split(X, y, test_size=0.05, random_state=7)   # → accuracy: 0.614
train_test_split(X, y, test_size=0.05, random_state=42)  # → accuracy: 0.891
```

- Маленькая тестовая выборка приводит к высокой дисперсии оценки.
- Непонятно, какому числу доверять.

Одно неудачное случайное разделение может существенно повлиять на итоговую оценку.

Shuffle=True vs False

Hold-Out

Плюсы:

- train и test имеют более похожее распределение;
- оценка модели обычно более стабильна.

Минус: нельзя использовать для временных рядов.

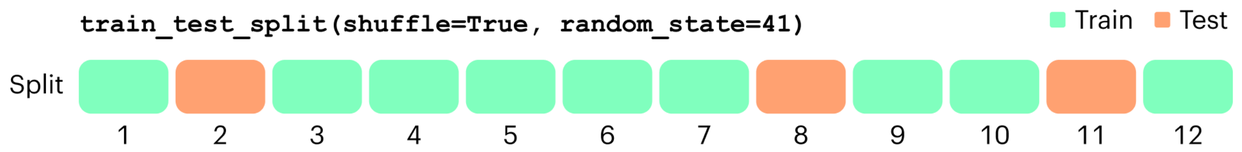
Плюс: подходит для временных рядов.

Минусы:

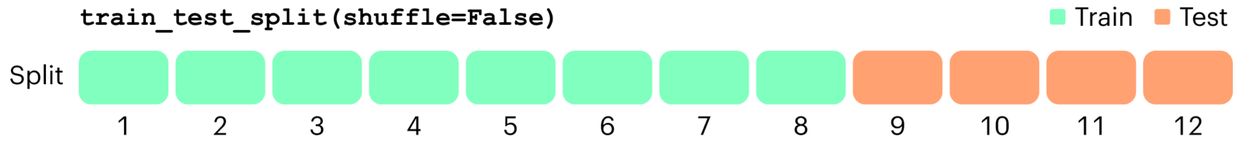
- результат может зависеть от порядка данных;
- возможен сдвиг распределений между train и test.

```
from sklearn.model_selection import train_test_split

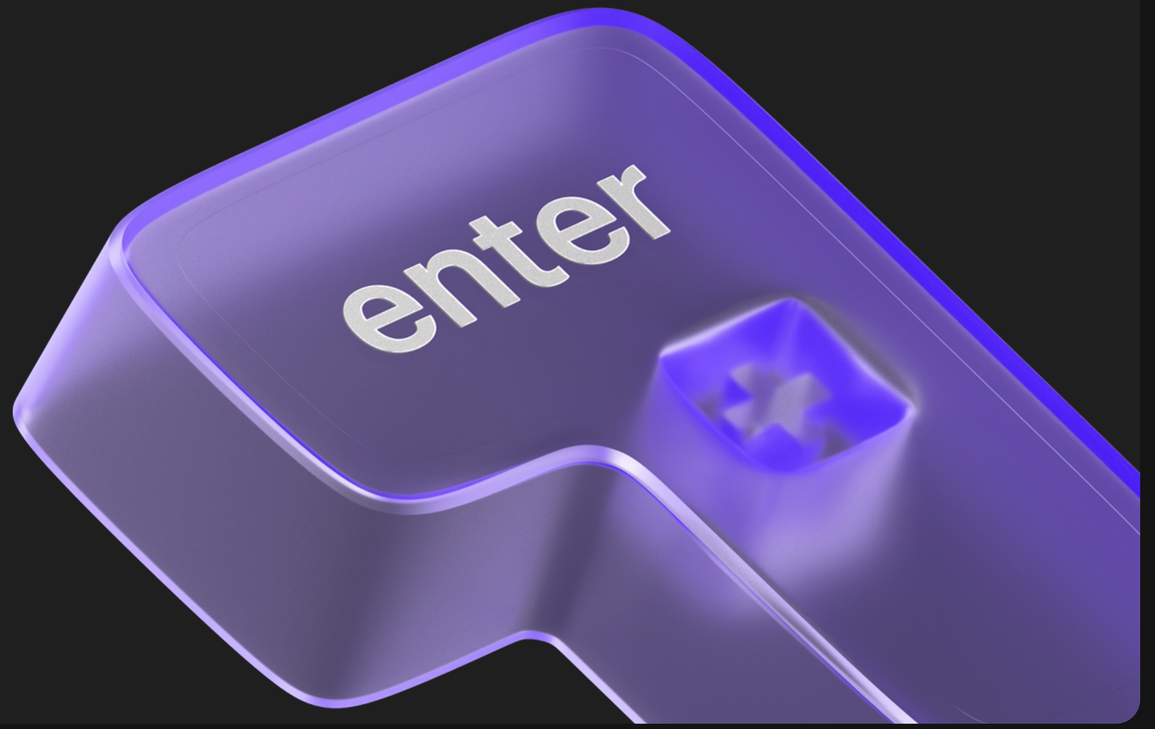
X_train, X_test, y_train, y_test = train_test_split (x, y,
test_size=0.33, random_state=42)
```




```
X_train, X_test, y_train, y_test = train_test_split
(x, y, test_size=0.33, shuffle=False)
```



Скользящий контроль



Кросс-валидация: K-Fold

K-Fold — более надёжная альтернатива простому Hold-Out, которая снижает зависимость оценки от одного конкретного разбиения. Модель проверяется на разных фолдах данных, что делает итоговую оценку более устойчивой и репрезентативной.

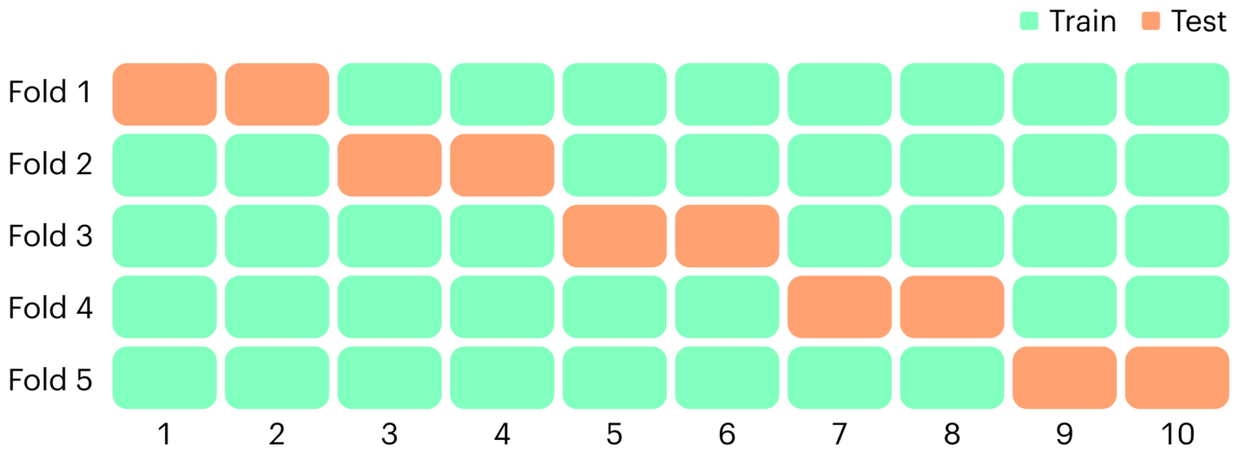


K-Fold кросс-валидация

1. Разделим выборку на k примерно равных частей (обычно k = 10).
2. Запускаем цикл по i = 1, ..., k:
   - i-я часть — для теста;
   - объединение остальных — для обучения.
3. Усредним k ошибок, вычисленных на разных итерациях цикла (на валидациях).

Можно использовать дисперсию для оценки доверия к полученному качеству.

```
sklearn.model_selection.KFold(n_splits=4,
                              shuffle=False,
                              random_state=None)
```

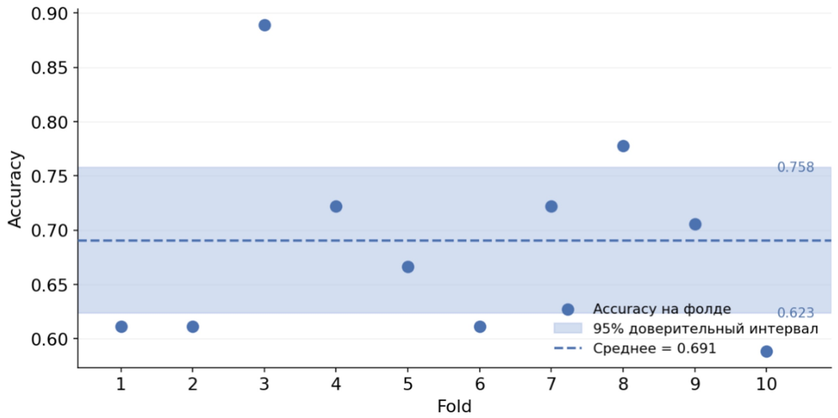


Построение доверительных интервалов

```
cv = KFold(n_splits=10, shuffle=True,
random_state=1)
scores =
cross_val_score(KNeighborsClassifier(5), X, y,
cv=cv)

mean = scores.mean()
sem = stats.sem(scores)
ci_low, ci_high = stats.t.interval(
    0.95, len(scores) - 1, loc=mean, scale=sem
)
```

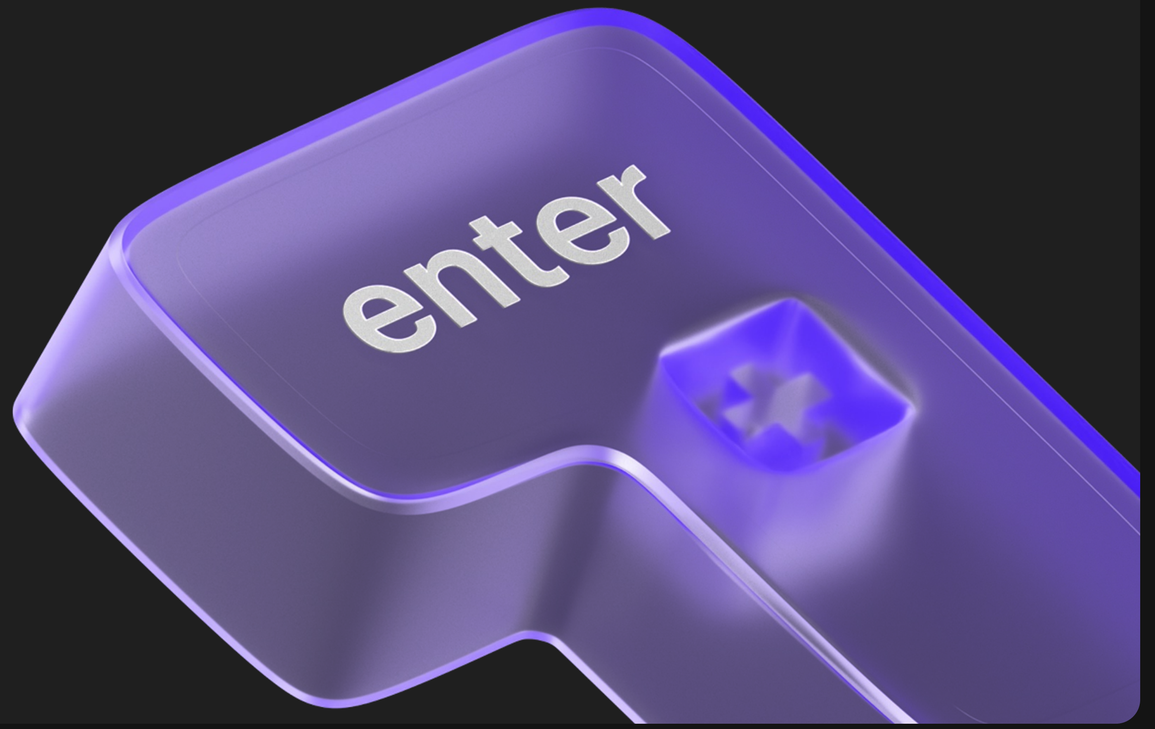
Доверительный интервал качества модели по результатам k-Fold



Фолды K-Fold коррелируют между собой, поэтому интервал получается уже, чем на самом деле. Для честной оценки лучше использовать Bootstrap, о котором поговорим позже.

Способы управления разбиением данных

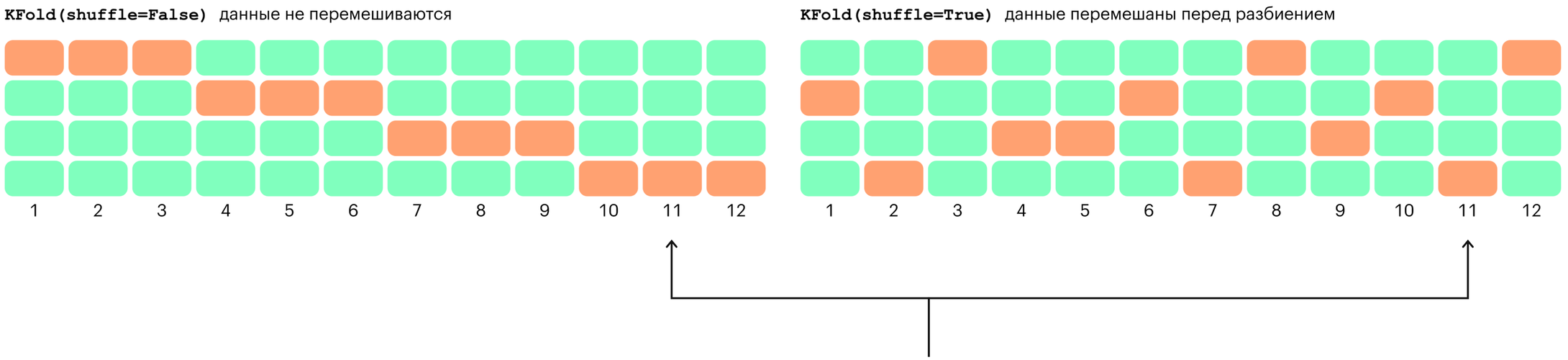
Shuffle, Group, Stratify



SHUFFLE

На примере K-Fold

Данные перемешиваются перед тем, как разделятся на k фолдов.



Каждый элемент в тестовую выборку попадает один раз в любом случае.

GROUP

Разбиение на выборки с учётом группировки данных

Иногда несколько объектов связаны между собой и **не должны попадать одновременно** в обучающую и тестовую выборки.

В этом случае данные разделяют **по группам**, а не по отдельным объектам.

Иначе скрытая утечка данных через группы!

G1	G1	G1	G2	G2	G3	G3	G3

Приведите пример задачи, где необходима группировка данных.

?

GROUP

Разбиение на выборки с учётом группировки данных

Предсказание по медицинским анализам

- 100 пациентов × 10 визитов = 1 000 строк.
- K-Fold (k = 5) → 200 строк случайно в каждый фолд.
- 7 визитов пациента Иванова → в train; 3 визита → в test.

Модель «видела» Иванова при обучении. При тесте на его данных она узнаёт «знакомое» — оценка завышена, на новых пациентах модель работает хуже.

Вывод: группируем по patient_id.

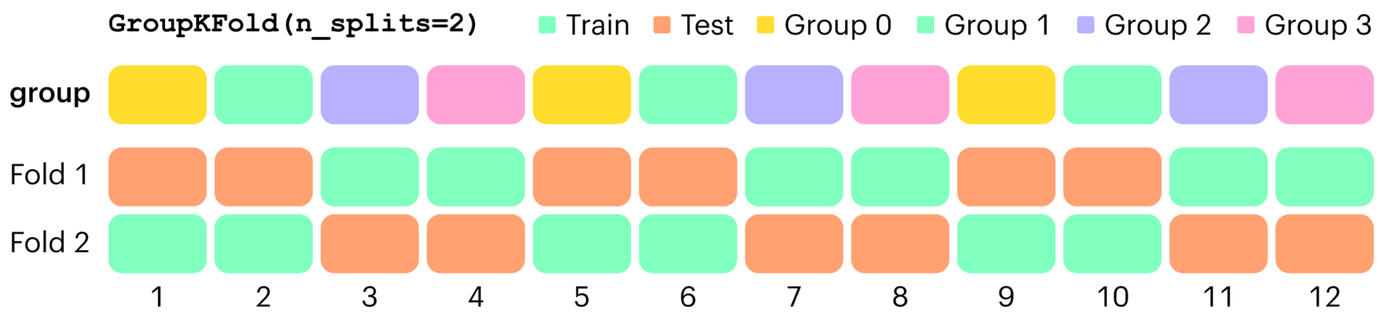
GROUP

На примере K-Fold

Каждая группа попадёт в тестовый фолд только один раз.

Количество уникальных групп должно быть не меньше количества фолдов.

```
sklearn.model_selection.GroupKFold(n_splits=2)
```



STRATIFY

Разбиение на выборки с учётом соотношения классов

Стратификация — это способ разделения данных, при котором сохраняется исходное соотношение классов в каждой выборке

Проблема: простое случайное разделение может изменить распределение данных. Особенно это важно в задачах классификации, где разные классы могут встречаться с разной частотой.

До разбиения

Общий датасет



90% класс A; 10% класс B

После разбиения

Train



90/10

Test



90/10

?

Приведите пример задачи, где необходима стратификация.

STRATIFY

Разбиение на выборки с учётом соотношения классов

Детектирование мошенничества

- 10 000 транзакций: 9 800 нормальных (98%) и 200 мошеннических (2%).
- K-Fold (k = 5) даёт в среднем 40 случаев фрода в каждом тестовом фолде.
- Но случайно может оказаться 12... 71... или вообще 0.

При нулевом фолде модель не видела примеров редкого класса при обучении → метрика бессмысленна или вводит в заблуждение

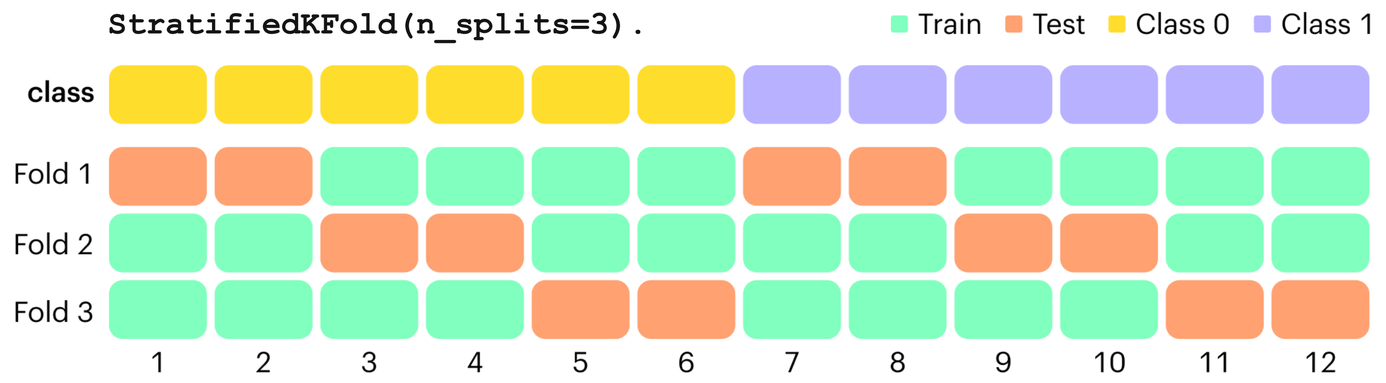
Стратификация особенно важна при несбалансированных классах, так как позволяет сохранить представительность редких классов в каждом фолде. Однако она применяется для любых задач классификации, когда важно сохранить распределение целевой переменной.

STRATIFY

На примере K-Fold

При создании фолдов сохраняется исходное распределение классов: доля каждого класса в каждом фолде остаётся примерно такой же, как в исходных данных.

```
sklearn.model_selection.StratifiedKFold(n_splits=3,
                                        shuffle=False,
                                        random_state=None)
```



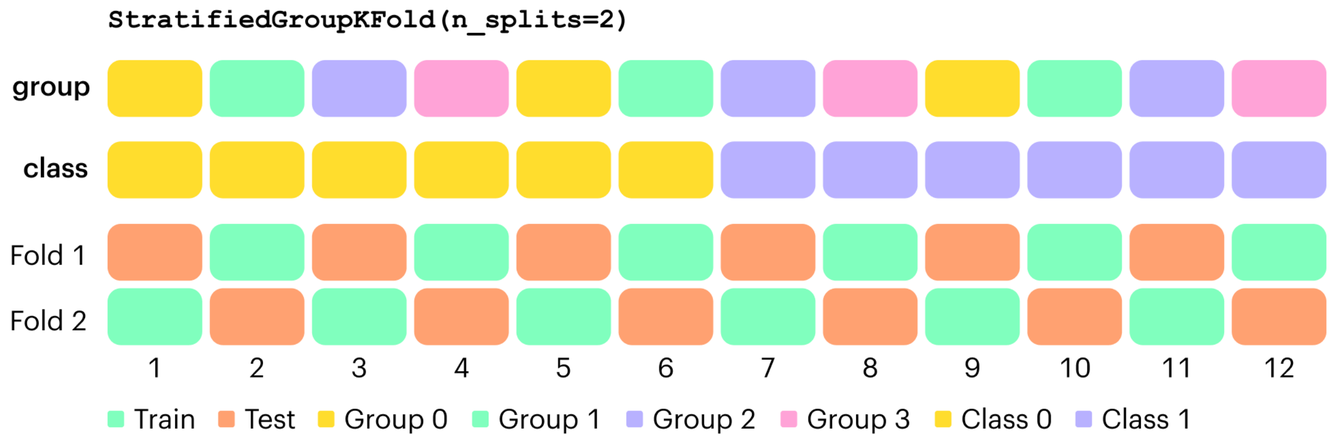
GROUP + STRATIFY

На примере K-Fold

Существует метод, который объединяет учёт групп и стратификацию.

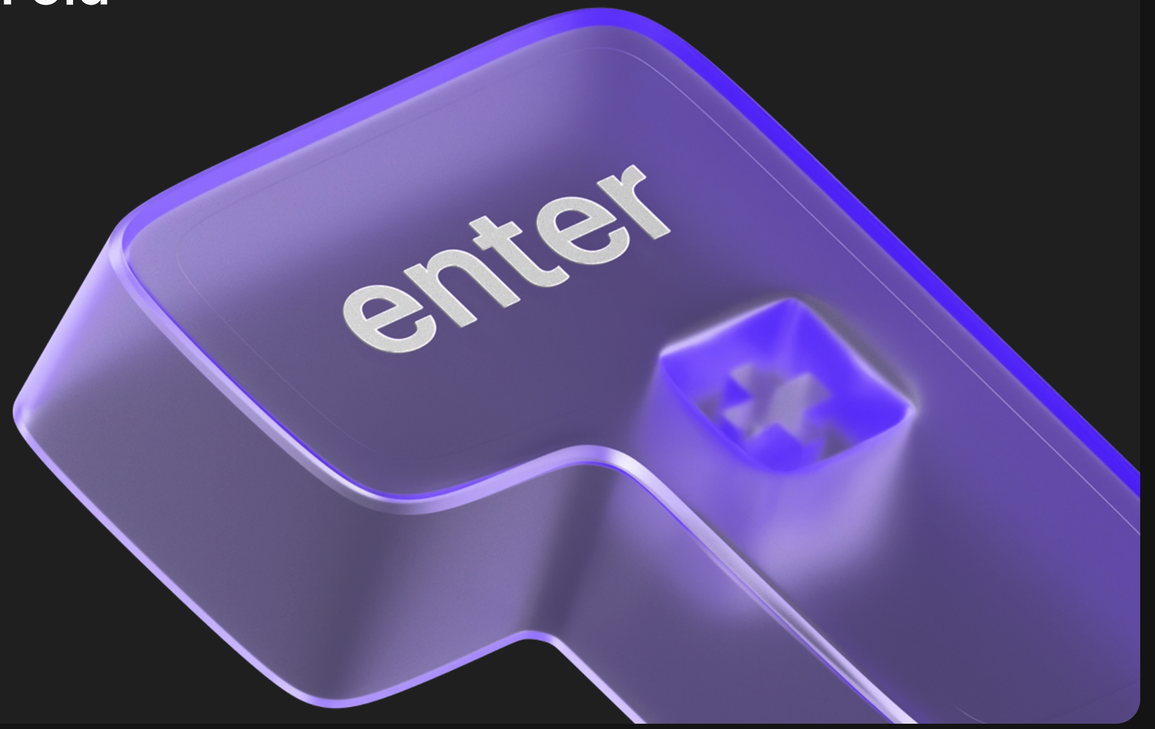
Разбивает данные по группам, стараясь сохранить распределение классов в каждом фолде настолько близким к исходному, насколько это позволяют границы групп.

```
sklearn.model_selection.StratifiedGroupKFold(n_splits=2,
                                             shuffle=False,
                                             random_state=None)
```



Другие методы кросс-валидации

ShuffleSplit, Leave-one-out, RepeatedKFold



ShuffleSplit

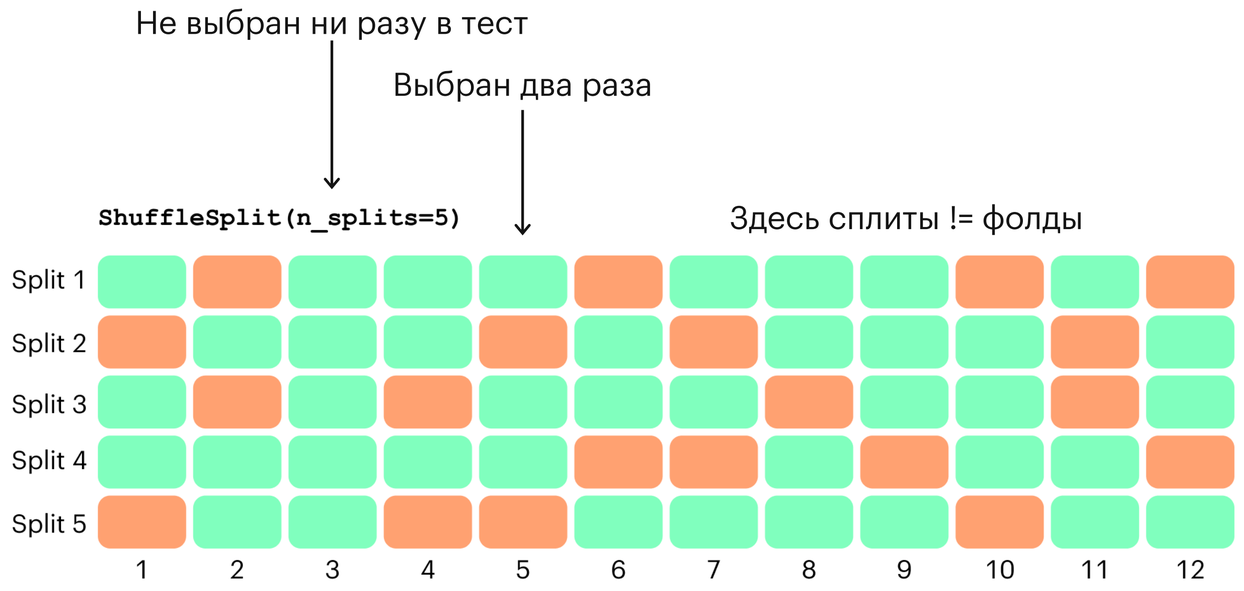
Random Subsampling Cross-Validation — это метод кросс-валидации, при котором данные **многократно случайным образом разделяются на обучающую и тестовую выборки.**

Мы k раз:

1. случайно выбираем отложенный контроль;
2. усредняем ошибки на всех отложенных выборках.

`train_test_split` — это функция-обёртка над `ShuffleSplit`.

```
sklearn.model_selection.ShuffleSplit(n_splits=4,
                                      test_size=0.3,
                                      train_size=None,
                                      random_state=42)
```

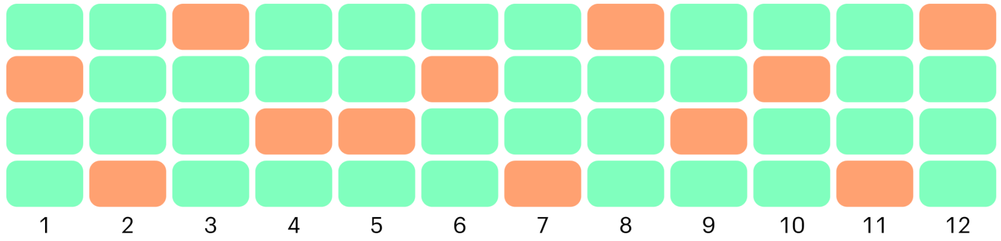


Отличие от K-Fold

K-Fold CV

- Делит данные на k фиксированных частей.
- Каждый объект попадает в тест ровно один раз (за один цикл).
- Все данные используются для тестирования.

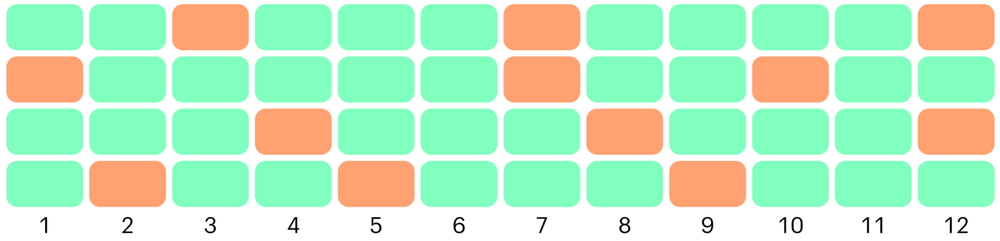
`KFold(shuffle=True)` данные перемешаны перед разбиением



Random Subsampling CV

- Каждый раз делает новое случайное разбиение.
- Один объект может попасть в тест несколько раз или ни разу.
- Только часть данных может быть использована.

`ShuffleSplit` независимые случайные разбиения



GROUP

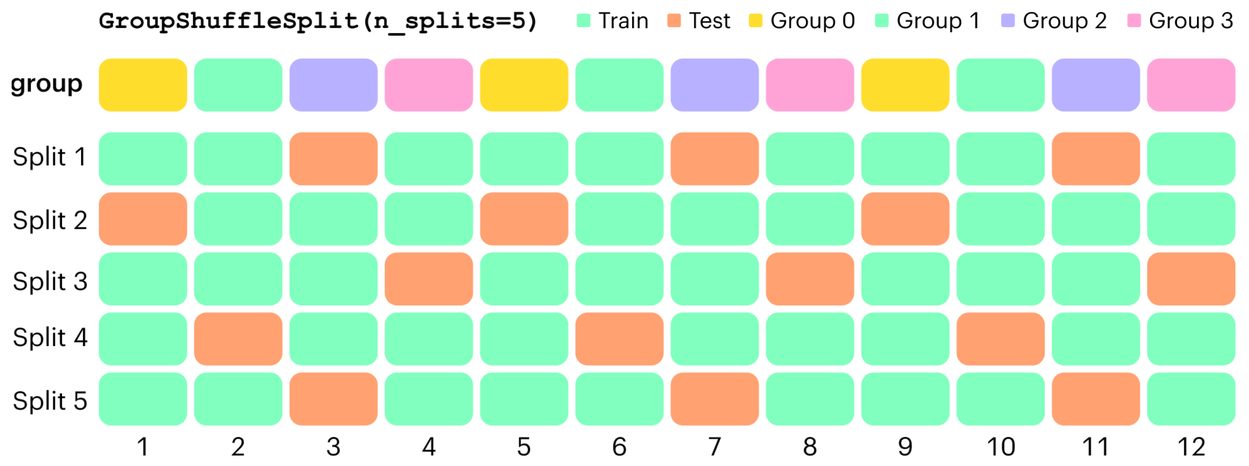
Random Subsampling

GroupShuffleSplit

Объекты из одной группы всегда попадают только в train или test.

Группы разбиваются случайно с заданной пропорцией, процесс можно повторить (n_splits раз).

```
sklearn.model_selection.GroupShuffleSplit(n_splits=4,
                                          test_size=0.3,
                                          train_size=None,
                                          random_state=42)
for t, (itrain, itest) in enumerate(cv.split(x, groups=g)):
    . . .
```



STRATIFY

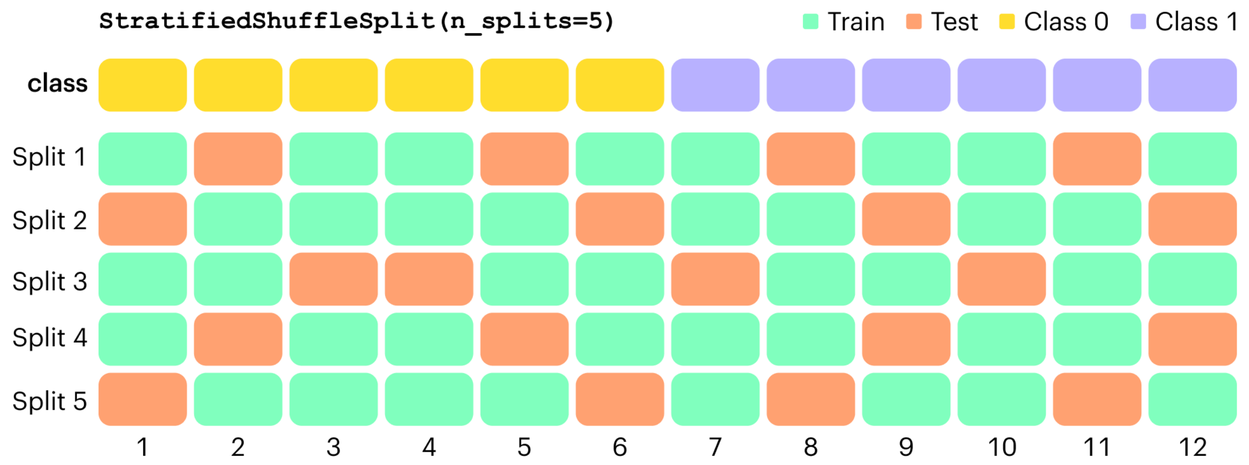
Random Subsampling

StratifiedShuffleSplit

Каждая выборка сохраняет исходное распределение классов (например, 90% класса 0 и 10% класса 1).

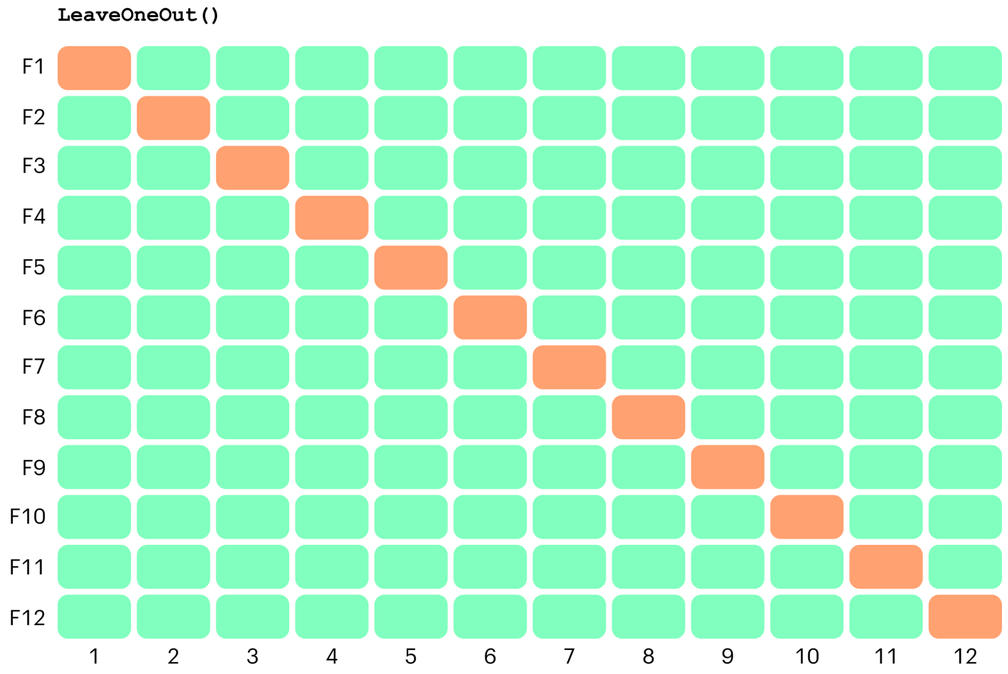
Разбиение выполняется случайно, но с контролем долей классов, процесс можно повторить несколько раз (n_splits).

```
sklearn.model_selection.StratifiedShuffleSplit(n_splits=4,
                                                test_size=0.3,
                                                train_size=None,
                                                random_state=None)
for t, (itrain, itest) in enumerate(cv.split(x, y)):
    ...
```



Leave-One-Out кросс-валидация

LOOCV



k-Fold при k = m, где m — мощность множества.

`sklearn.model_selection.LeaveOneOut`

Может слишком долго вычисляться.

Каждое обучение
использует n-1 объектов

Ещё есть **`sklearn.model_selection.LeavePOut`**.

Он создаёт разбиение по всем
возможным P-объектам.

Карта методов

	Без стратификации	Со стратификацией
Без группировки	`train_test_split`, `KFold`, `LeaveOneOut`, `ShuffleSplit`, `RepeatedKFold`	`StratifiedKFold`, `StratifiedShuffleSplit`
С группировкой	`GroupKFold`, `GroupShuffleSplit`, `LeaveOneGroupOut`	`StratifiedGroupKFold`

Порассуждаем

Какой/какие методы лучше выбрать для этих задач?

Предсказание заболеваний

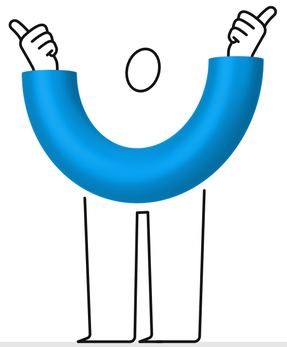
30 пациентов в 5 клиниках (по 6 в каждой).
У каждого по 4 лабораторных визита, итого 120 строк.
Положительный исход у 9 из 30 (30%).

Предсказание патологий

20 000 рентгеновских снимков (каждый — отдельный пациент).
1% с редкой патологией — 200 случаев.
Нужно сравнить 3 алгоритма. Одно обучение = 2 часа.
Бюджет: 15 часов.

Кредитный скоринг

500 клиентов, у каждого ровно 20 кредитных заявок (10 000 строк). Дефолты у 50 клиентов из 500 (10%) = = 1 000 дефолтных строк. Нужно, чтобы модель обобщалась на новых клиентов.



Порассуждаем

Какой/какие методы лучше выбрать для этих задач?

Предсказание заболеваний

StratifiedGroupKFold(n_splits=5) (группа = пациент)

Все визиты одного пациента попадают целиком либо в train, либо в test, поэтому модель оценивается на новых пациентах без утечки данных.

Стратификация помогает сохранить долю положительных исходов: в каждом тестовом фолде будет примерно 6 пациентов, из них 1–2 с положительным исходом.

Предсказание патологий

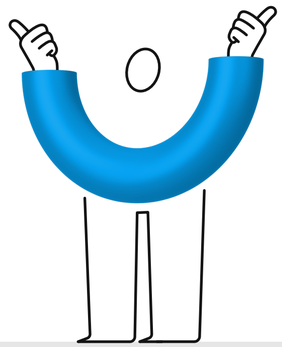
```
StratifiedShuffleSplit(n_splits = 2, test_size = 0.2)
```

2 × 3 × 2 = 12 часов и ровно 40 патологий в каждом тесте гарантировано.

Кредитный скоринг

```
StratifiedGroupKFold(n_splits = 5)
```
(группа = клиент)

StratifiedKFold: заявки одного клиента в train и test одновременно — утечка. GroupKFold (k = 5): 500 ÷ 5 = = 100 клиентов в каждом тесте, но дефолтных может случайно попасть 6 или 14 вместо ровных 10 — нестабильно. StratifiedGroupKFold: ровно 100 клиентов в тест и ровно 10 дефолтных в фолде.



Три золотых правила разбиения выборки

01

Тест или валидация должны моделировать реальную работу алгоритма.

- Деление по группам
- Предсказание будущего
- Стратификация **`stratify = y`**

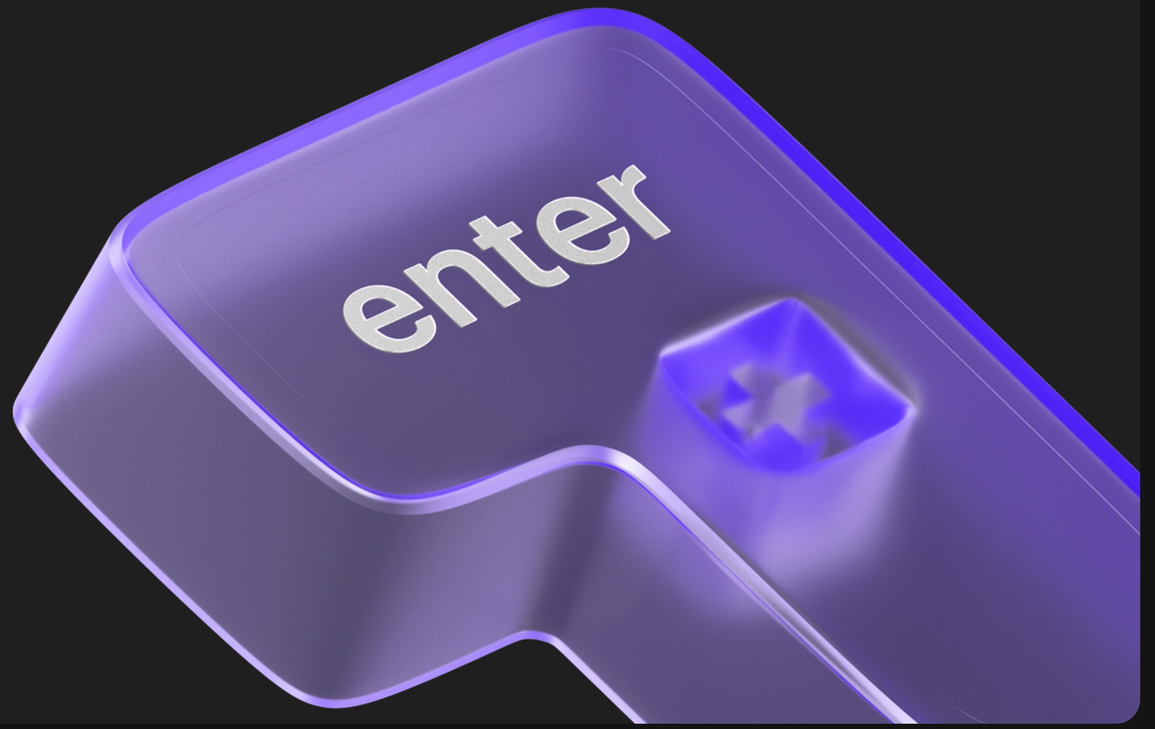
02

Тест или валидация должны быть случайным (**`shuffle = True`**) или специально подготовленным (убирать дубликаты).

03

Нельзя явно или неявно использовать метки объектов, на которых оцениваем ошибку (качество). Нужно делать корректно селекцию признаков.

Бутстрэп
Bootstrap



Бутстрэп

Bootstrap

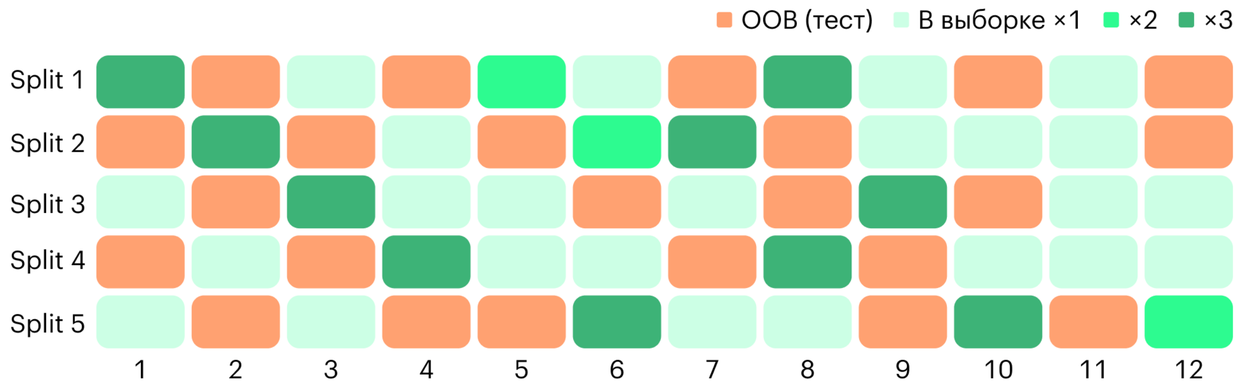
Bootstrap — это техника генерации новых выборок из исходных данных путём случайного отбора объектов с возвращением.

Bootstrap позволяет оценить не только среднее значение метрики качества, но и её разброс, что даёт возможность построить доверительные интервалы и оценить статистическую надёжность результатов.

В sklearn нет готовой реализации.

```
from sklearn.utils import resample
import numpy as np

train = resample(indices, replace=True,
n_samples=n, random_state=i)
test  = np.setdiff1d(indices, train)
```



Бутстрэп

Bootstrap

Алгоритм

1. Формируем bootstrap-выборку.
   - Случайно выбираем m объектов с возвращением.
   - Некоторые объекты попадут несколько раз.
   - Около 63% объектов окажутся в train хотя бы один раз.
   - Около 37% вообще не попадут.
2. Обучаем модель на bootstrap-выборке.
3. Тестируем модель на объектах, которые не попали в bootstrap (они называются Out-of-Bag, OOB).

В контроль попадает Out-of-Bag: получаем примерно следующее значение.

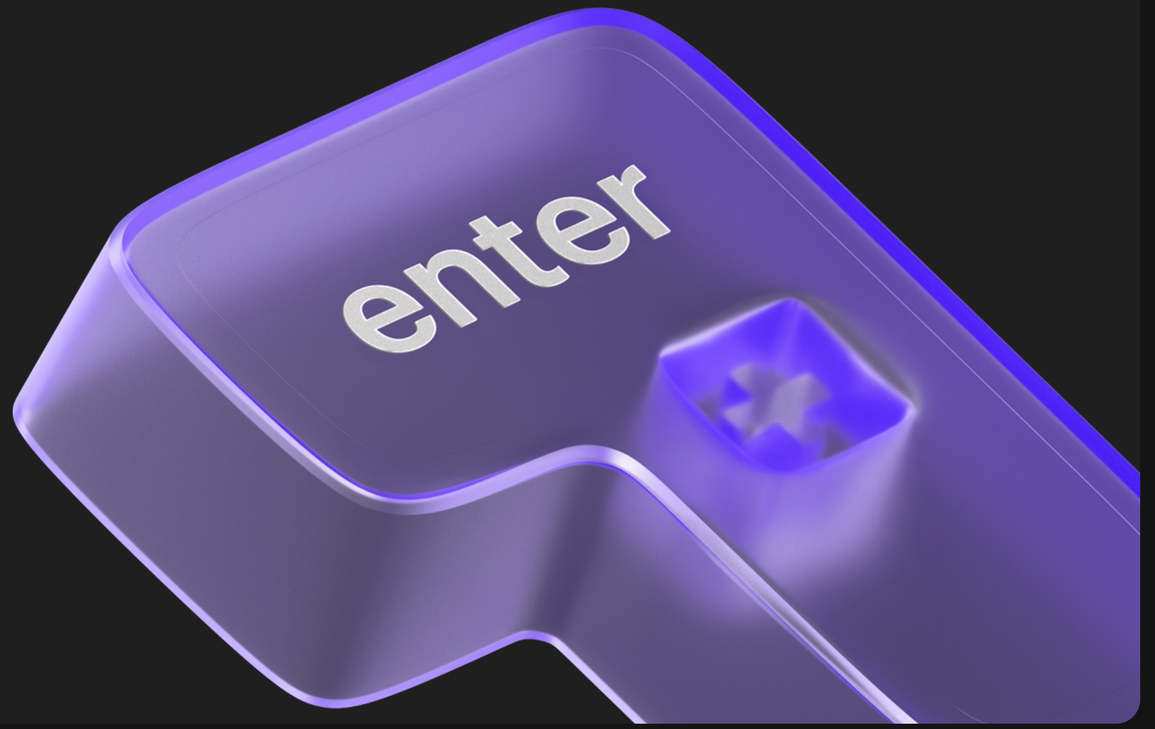
$$\left(1-\frac{1}{m}\right)^m \approx e^{-1} \approx 0.37 = 37\% \text{ выборки}$$

Модель учится на выборке (которую мы построим по всему доступному датасету) того же объёма, что и итоговая.

- Использует не все данные.
- Есть дубликаты.

С точки зрения распределения бутстрэп-выборка похожа на исходную.

Контроль по времени
Out-of-Time-контроль



Оценка качества модели на временных рядах

Временные ряды — отдельный случай: нам нельзя допускать утечек из будущего в прошлое.

Самый базовый подход, который можно использовать, — последовательный Hold-Out, при котором данные выборки разбиваются по временной шкале.

Train: ранние периоды (например, 2020–2022 годы).

Test: самые последние наблюдения (например, 2023–2024 годы).

Expanding Window

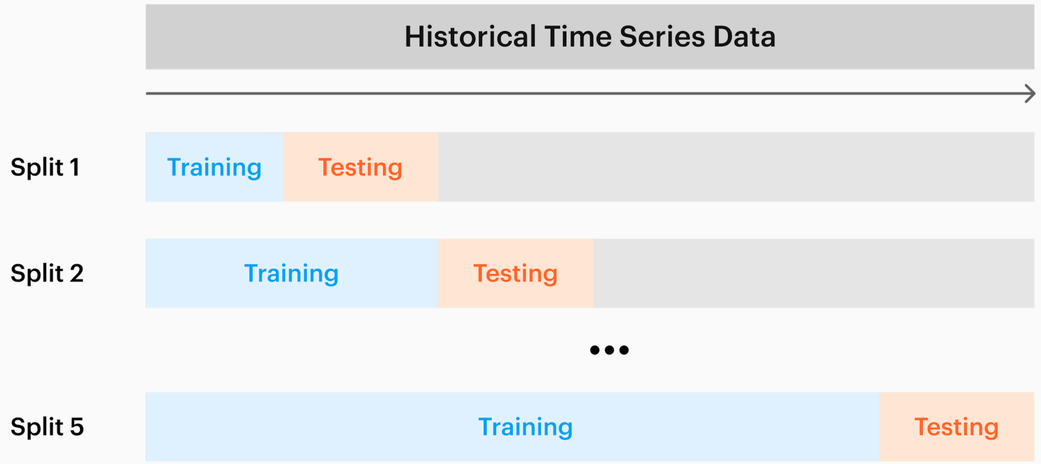
Нарастающее окно (англ. expanding window) — способ валидации модели,

- при котором на каждом шаге Train-окно расширяется;
- в Validation всегда попадает следующий временной блок.

Такой метод подходит для задач, где важно учитывать весь исторический контекст.

ПРИМЕР

- Fold 1: train [2019], val [2020].
- Fold 2: train [2019–2020], val [2021].
- Fold 3: train [2019–2021], val [2022].



Sliding Window

Скользящее окно (англ. sliding window) — ещё один метод работы при валидации на временных рядах, при котором train-окно фиксировано по размеру, но «скользит» вперёд по временной шкале вместе с валидацией.

Такой метод подходит для случаев, в которых старые данные со временем теряют актуальность.

ПРИМЕР

- Fold 1: train [2019–2020], val [2021].
- Fold 2: train [2020–2021], val [2022].
- Fold 3: train [2021–2022], val [2023].



Контроль по времени

Out-of-Time-контроль

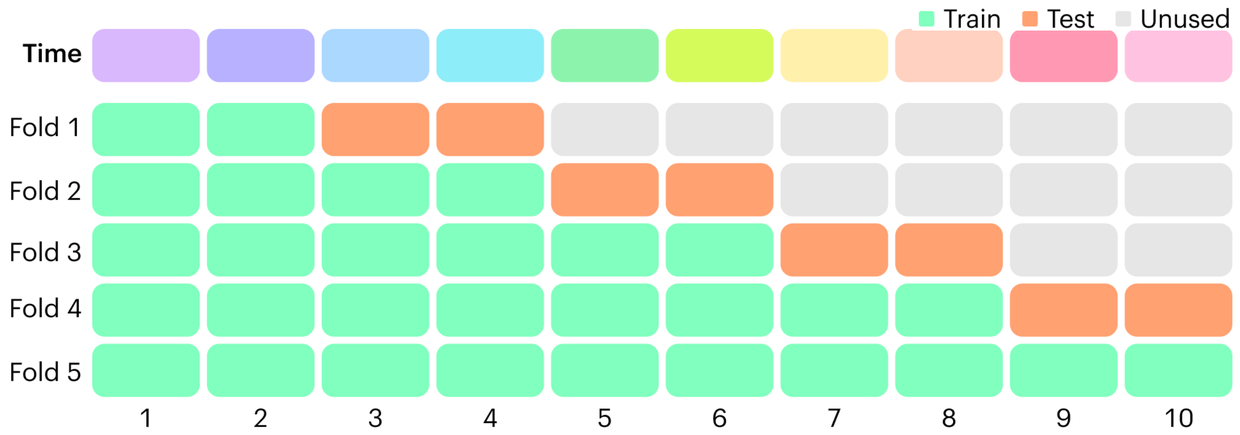
TimeSeriesSplit — функция для разбиения временных рядов с целью кросс-валидации.

С параметрами по умолчанию реализуется метод **Expanding Window**.

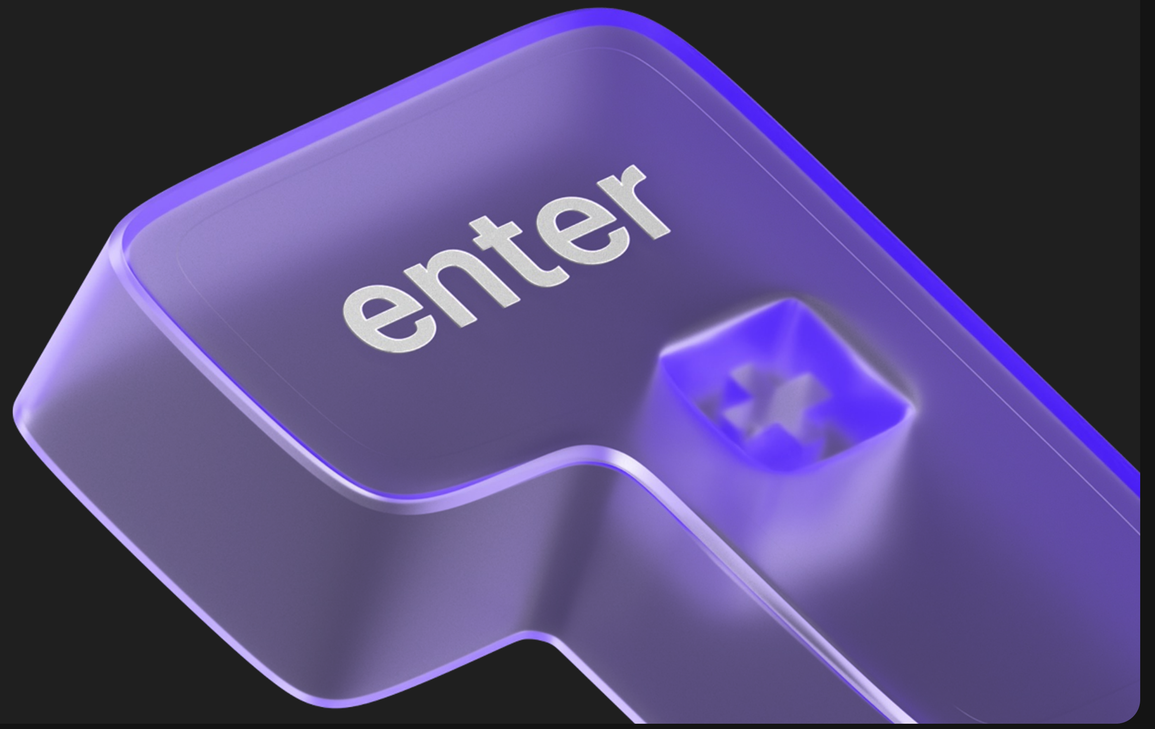
Для реализации **Sliding Window** нужно задать **`max_train_size`**, который выступит размером скользящего окна.

Параметр **`gap`** отвечает за пропуск нескольких последних наблюдений перед тестовой выборкой, чтобы данные из train не были слишком близки к test.

```
sklearn.model_selection.TimeSeriesSplit(n_splits=5,
                                        max_train_size=None,
                                        test_size=None,
                                        gap=0)
```



Подбор гиперпараметров



Проблема валидации и тестирования

Мы обсуждали проблему оценки качества алгоритмов.

Для её решения требуется корректный выбор модели (Model Selection).

Мы хотим достичь наименьшего значения ошибки итогового решения через выбор модели (в широком смысле — «пайплайна»), а именно через выбор:

- → модели алгоритмов
- → значения гиперпараметров
- → подходящих признаков
- → способа предобработки данных

Ключевые определения

→ **Модель (англ. model)** — параметрическое семейство алгоритмов.

→ **«Обычные» параметры (англ. model parameters)** настраиваются в результате обучения.

→ **Гиперпараметры (англ. hyperparameters)** выбираются до обучения экспертно / настраиваются переборно.

Количество возможных моделей конечно, но чрезвычайно велико. Например:

- kNN
- NearestCentroid
- SVM
- Linear
- NN

Число алгоритмов практически неограниченно (бесконечно в теоретическом смысле). Например:

- 1NN, 3NN, 5NN ...
- SVM(C=0.27), SVM(C=0.15) ...

Модификация основной идеи для выбора и тестирования

Обучающая выборка (Training Set)

Обучение модели (настройка её параметров)

Валидационная выборка (Validation Set)

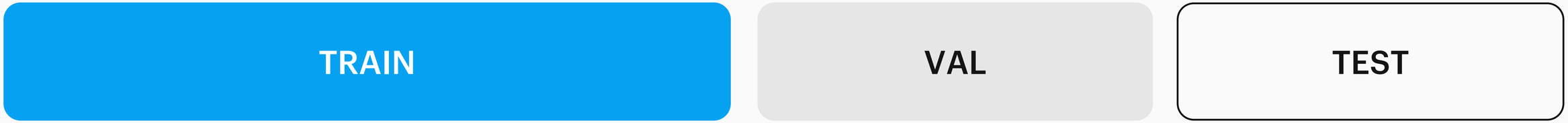
Выбор пайплайна (модели/гиперпараметров/признаков)

Иногда — локальный контроль

Тестовая выборка (Test Set)

Оценка качества алгоритма

Иногда — итоговая оценка



Модификация основной идеи для выбора и тестирования

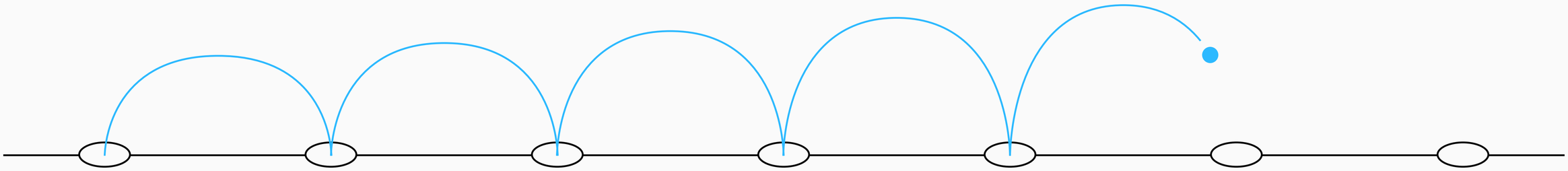


Можно проводить и более сложные схемы, например: k-Fold CV (валидация) + Hold-Out (итоговый тест).

В редких случаях доверяют результатам валидации. Для уверенной оценки всегда нужна Test-выборка.

Где используется выбор CV-контроля

- → Оценка качества модели
- → Настройка (перебор значений) гиперпараметров
- → «Честные» ответы: как бы алгоритм работал без обучения на этих данных
- → Контроль алгоритмов
- → Анализ качества модели через кривые валидации и обучения:
  - Validation Curves (от значений)
  - Learning Curves (от объёма выборки)
- → Предварительная обработка (позже ещё и включая кодирование категорий)



Применим на практике

Оценка модели с помощью выбранного контроля

```
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import ShuffleSplit
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression()

cv = ShuffleSplit(n_splits=3, test_size=0.1,
                  train_size=None,random_state=42)

cross_val_score(logreg, X, y, cv=cv)
```

Эти функции имеют множество параметров (они взаимосовместимы).

По умолчанию используется встроенный в модель скорер.

Есть аналогичная функция, сохраняющая время обучения и работы:

sklearn.model_selection.cross_validate

Перестановочный тест для оценки неслучайности результата:

sklearn.model_selection.permutation_test_score

Перебор значений гиперпараметров

Делим данные на обучение и контроль (возможно многократное повторение).
При разных значениях параметров обучаем алгоритм и проверяем его качество.

```
from sklearn.model_selection import GridSearchCV
parameters = {'metric':('euclidean', 'manhattan',
'chebyshev'), 'n_neighbors':[1, 3, 5, 7, 9, 11]}

clf = GridSearchCV(estimator, parameters, cv=5,
scoring='roc_auc'})

clf.fit(X, y)
clf.cv_results_['mean_test_score']
```

	k = 1	k = 3	k = 5	k = 7	k = 9	k = 11
euclidean	76.0	77.0	79.0	78.5	80.5	82.5
manhattan	74.0	74.0	79.0	79.5	80.5	81.0
chebyshev	76.5	78.5	80.0	80.0	81.0	81.5

Можно также использовать случайный поиск:
model_selection.RandomizedSearchCV

В нём можно настроить
«число итераций» — передавать
распределения параметров.

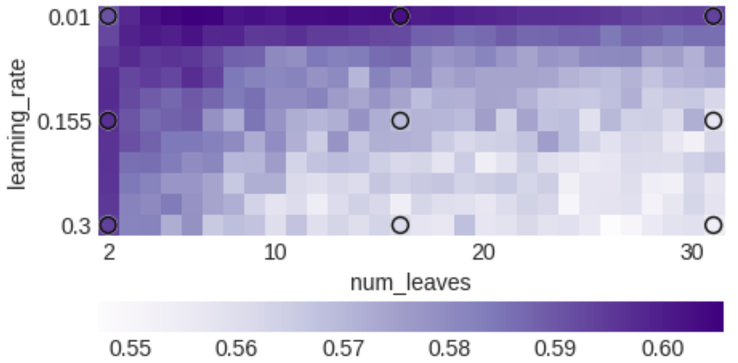
Перебор параметров

Случайный поиск считается наиболее предпочтительным.

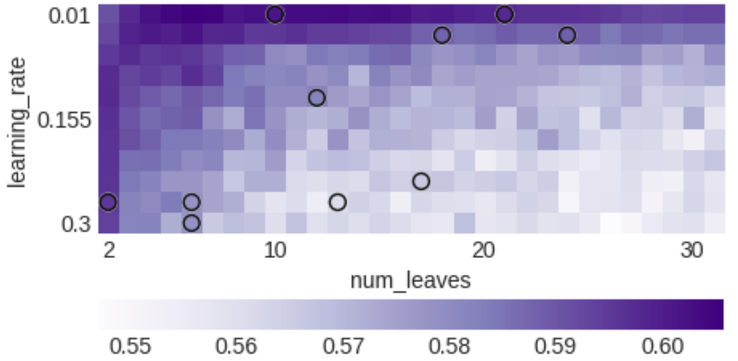
```
import lightgbm as lgb
model = lgb.LGBMClassifier(n_estimators=100,
subsample=0.75, colsample_bytree=0.75)

from sklearn.model_selection import GridSearchCV
parameters = {'num_leaves': np.arange(2, 32),
              'learning_rate':np.linspace(0.01, 0.3,
11)}
clf = GridSearchCV(model, parameters, cv=5,
scoring='roc_auc')
clf.fit(X, y)
```

GridSearchCV

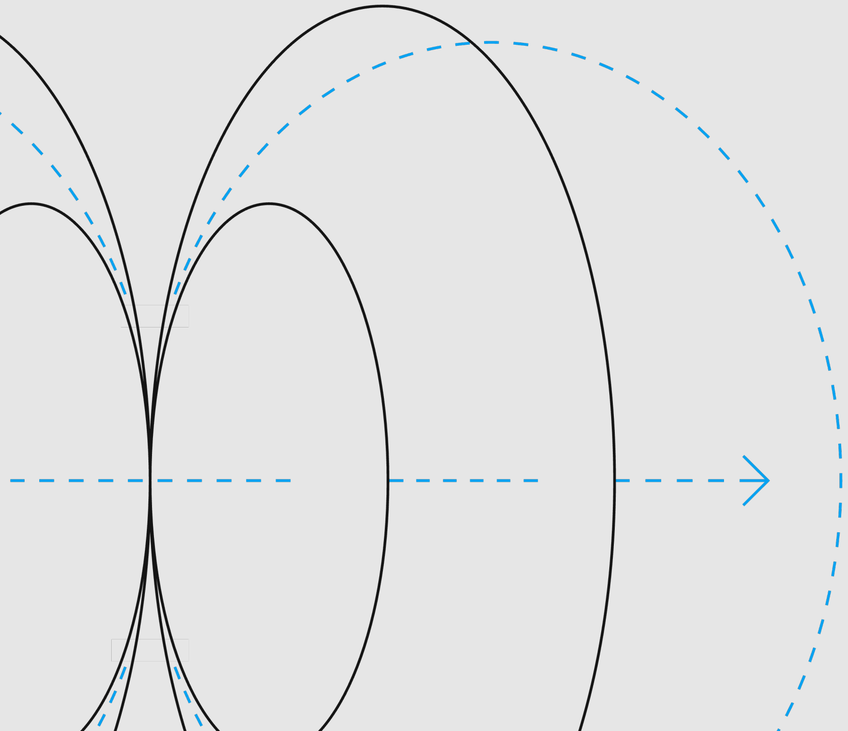


RandomizedSearchCV



Применим на практике

Как получить параметры модели



```
model.get_params()
{'boosting_type': 'gbdt',
 'class_weight': None,
 'colsample_bytree': 0.75,
 'importance_type': 'split',
 'learning_rate': 0.1,
 'max_depth': -1,
 'min_child_samples': 20,
 'min_child_weight': 0.001,
 'min_split_gain': 0.0,
 'n_estimators': 100,
 'n_jobs': -1,
 'num_leaves': 31,
 'objective': None,
 'random_state': None,
 'reg_alpha': 0.0,
 'reg_lambda': 0.0,
 'silent': True,
 'subsample': 0.75,
 'subsample_for_bin': 200000,
 'subsample_freq': 0}
```

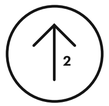
Optuna: автоматический подбор гиперпараметров



Байесовская оптимизация. Строит модель вероятности «хороших» параметров, тем самым не перебирает вслепую, а предсказывает оптимальные.



Pruning (Early Stopping). Автоматически обрывает неперспективные попытки до конца, что экономит вычисления в разы.



Мультицелевая оптимизация. Optuna поддерживает Pareto-фронты — точки, которые не хуже одновременно по всем метрикам.



Гибкость и интеграции. XGBoost, LightGBM, CatBoost, PyTorch, sklearn, распределённые вычисления из коробки.

```python
import optuna

# Шаг 1. Создаём study для хранения результатов поиска
study = optuna.create_study(direction="maximize",
sampler=optuna.samplers.TPESampler(seed=42))

# Шаг 2. Запускаем подбор гиперпараметров
study.optimize(objective, n_trials=50)

# Шаг 3. Получаем лучший результат
print("Лучшие параметры:", study.best_params)
print("Лучший ROC-AUC:", study.best_value)

# Шаг 4. Анализируем процесс оптимизации
optuna.visualization.plot_optimization_history(study).show(
)
optuna.visualization.plot_param_importances(study).show()
```

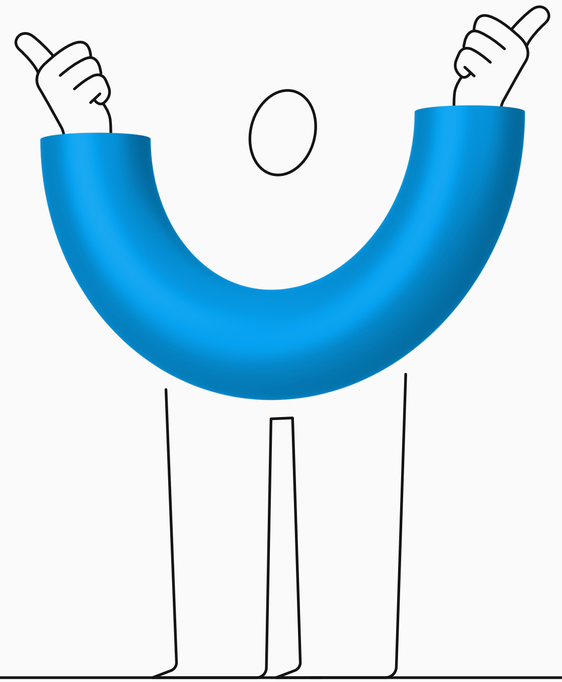
Подбор гиперпараметров

Метод	Принцип работы	Где применим	Плюсы	Минусы
Grid Search	Полный перебор по сетке	Маленькие пространства, Baseline	Простота, воспроизводимость	Экспоненциальный рост времени при увеличении числа параметров
Randomized Search	Случайный выбор комбинаций	Большие пространства, быстрый прототип	Быстрее, чем Grid, при ограниченном бюджете	Риск неоптимального разбиения
Hyperopt	Байесовская оптимизация (TPE)	Модели со сложными иерархиями параметров	Эффективен, гибок, Open Source	Относительно сложный метод с устаревшим интерфейсом и ручной визуализацией
Optuna	Байесовская оптимизация + Pruning	Современные ML-задачи, Kaggle, продакшен	Поддержка Early Stopping, многофункционален, гибок	Требует начальной настройки

Кампус Центрального университета. Источник

Советы

- Не забывай указывать метрику качества (а лучше несколько): **`score`**
- Распараллеливание: **`n_jobs=-1`**
- Можно сделать вычисления устойчивым к ошибкам: **`error_score=0`** (но аккуратно!)
- Оптимизировать целый пайплайн
- Использовать современные методы, например Optuna!



Итоги лекции

Существует множество типов контроля качества ML-моделей. Чаще всего их объединяют в единую группу, которая называется **кросс-валидацией**.

Основа любого вида контроля — **разбиение** данных на несколько частей.

Среди видов кросс-валидации:

- Hold-Out
- k-Fold
- Leave-One-Out
- Stratified k-Fold

Бутстрэп — способ формирования новых выборок из имеющихся данных.

Кросс-валидация на временных рядах должна быть реализована так, чтобы не допустить утечек из будущего в прошлое.

Для подбора гиперпараметров данные разбивают как минимум на 3 части.

Гиперпараметр — настраиваемый параметр модели, который эксперты задают до обучения.

Для подбора гиперпараметров существуют автоматизированные системы. На практике стоит пользоваться в основном современными методами.

ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ



Вопросы

